Mise à l'échelle

CSI4506 Introduction à l'intelligence artificielle

Marcel Turcotte

2026-10-13

Ce carnet exécutable de façon autonome approfondit l'exemple de mise à l'échelle de la leçon 10. Il montre comment l'échelle des attributs modifie la géométrie employée par les méthodes fondées sur les plus proches voisins.

Scénario

Considérons la prédiction du prix d'une maison par **régression des k plus proches voisins (KNN)** à partir de deux attributs :

- x_1 : nombre de pièces (petite échelle)
- x_2 : superficie en pieds carrés (grande échelle)

Les trois exemples **a**, **b** et **c** sont choisis de façon à ce que :

- **sans mise à l'échelle**, **a** soit plus proche de **b**, car la superficie domine ;
- **avec une standardisation (score z)**, **a** devienne plus proche de **c**, car la différence du nombre de pièces compte une fois les attributs remis à l'échelle.

Données (trois maisons)

```
import numpy as np
import pandas as pd

# Trois exemples (pièces, pieds carrés); prix pour b et c seulement
point_names = ["a", "b", "c"]
X = np.array([
    [4, 1500.0],  # a (requête)
    [8, 1520.0],  # b (entraînement)
```

```
    [4, 1300.0],  # c (entraînement)
], dtype=float)

prices = pd.Series([np.nan, 520_000, 390_000], index=point_names, name="prix")

df = pd.DataFrame(X, columns=["pièces", "pieds_carrés"], index=point_names)
display(df)
display(prices.to_frame())
```

	pièces	pieds_carrés
a	4.0	1500.0
b	8.0	1520.0
c	4.0	1300.0

	prix
a	NaN
b	520000.0
c	390000.0

Les exemples **b** et **c** forment l'ensemble d'entraînement. L'exemple **a** constitue la requête dont nous voulons prédire le prix. Ce jeu de données volontairement minuscule illustre la géométrie, et non l'estimation d'un modèle réaliste.

Distances euclidiennes (sans mise à l'échelle)

La distance euclidienne au carré entre u et v est

$$\|u - v\|_2^2 = \sum_j (u_j - v_j)^2.$$

La distance euclidienne est sa racine carrée, $\|u - v\|_2$; les deux expressions classent les voisins dans le même ordre.

Lorsqu'un attribut possède une échelle beaucoup plus grande, comme la superficie, il peut dominer la somme.

```python
from sklearn.metrics import pairwise_distances

dist_unscaled = pd.DataFrame(
    pairwise_distances(df.values, metric="euclidean"),
    index=df.index, columns=df.index
)
dist_unscaled
```

	a	b	c
a	0.000000	20.396078	200.000000
b	20.396078	0.000000	220.036361
c	200.000000	220.036361	0.000000

```python
print("Plus proche de 'a' (sans mise à l'échelle) :", dist_unscaled.loc["a"].drop("a").idxmi
```

```
Plus proche de 'a' (sans mise à l'échelle) : b
```

Résultat : **a** est plus proche de **b**, car la différence de superficie domine la distance.

Mise à l'échelle appropriée pour la modélisation

Pour éviter toute fuite de données (*data leakage*), calculons les paramètres de mise à l'échelle à partir des seules données d'entraînement (**b** et **c**), puis transformons les données d'entraînement et la requête :

$$z(x) = \frac{x - \mu_{\text{entraînement}}}{\sigma_{\text{entraînement}}}.$$

```python
from sklearn.preprocessing import StandardScaler

train_idx = ["b", "c"]
query_idx = ["a"]

scaler = StandardScaler()

scaler.fit(df.loc[train_idx])     # ajuster uniquement sur l'entraînement

Z = pd.DataFrame(
    scaler.transform(df),
```

```
    columns=df.columns, index=df.index
)

Z
```

	pièces	pieds_carrés
a	-1.0	0.818182
b	1.0	1.000000
c	-1.0	-1.000000

Distances euclidiennes (après mise à l'échelle)

```
dist_scaled = pd.DataFrame(
    pairwise_distances(Z.values, metric="euclidean"),
    index=Z.index, columns=Z.index
)
dist_scaled
```

	a	b	c
a	0.000000	2.008247	1.818182
b	2.008247	0.000000	2.828427
c	1.818182	2.828427	0.000000

```
print("Plus proche de 'a' (après mise à l'échelle) :", dist_scaled.loc["a"].drop("a").idxmin
```

```
Plus proche de 'a' (après mise à l'échelle) : c
```

Nouveau résultat : **a** est plus proche de **c**, car la différence du nombre de pièces compte une fois les attributs placés sur des échelles comparables.

Régression KNN : changement de prédiction

Un régresseur 1-NN prédit le prix de l'exemple d'entraînement le plus proche. Comparons le même estimateur **avec et sans** mise à l'échelle.

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline

X_train = df.loc[train_idx].values          # b, c
y_train = prices.loc[train_idx].values      # prix de b et c
X_query = df.loc[query_idx].values          # a

# 1) Sans mise à l'échelle
knn_plain = KNeighborsRegressor(n_neighbors=1, metric="euclidean")
knn_plain.fit(X_train, y_train)
pred_plain = knn_plain.predict(X_query)[0]

# 2) Avec mise à l'échelle; le pipeline ajuste le scaler sur l'entraînement
knn_scaled = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=1, metric="euclidean"))
])
knn_scaled.fit(X_train, y_train)
pred_scaled = knn_scaled.predict(X_query)[0]

pd.DataFrame(
    {
        "prédiction (sans mise à l'échelle)": [pred_plain],
        "prédiction (avec mise à l'échelle)": [pred_scaled],
        "plus proche voisin (sans mise à l'échelle)": [
            train_idx[
                knn_plain.kneighbors(
                    X_query, return_distance=False
                )[0, 0]
            ]
        ],
        "plus proche voisin (avec mise à l'échelle)": [
            train_idx[
                knn_scaled["knn"].kneighbors(
                    knn_scaled["scaler"].transform(X_query),
                    return_distance=False,
                )[0, 0]
            ]
        ],
    },
    index=["a"]
)
```

	prédiction (sans mise à l'échelle)	prédiction (avec mise à l'échelle)	plus proche voisin (sans mise à l'éche
a	520000.0	390000.0	b

À retenir :

- **Sans mise à l'échelle :** a b, donc une prédiction d'environ **520 000 $**
- **Avec mise à l'échelle :** a c, donc une prédiction d'environ **390 000 $**

La cible de la requête est inconnue. Il ne s'agit donc pas d'une comparaison de performance. L'exemple montre que la représentation choisie a modifié le plus proche voisin et, par conséquent, la prédiction.

Pourquoi ce changement ?

- La distance euclidienne au carré additionne les différences au carré de chaque attribut :

$$\|u - v\|_2^2 = \sum_j (u_j - v_j)^2.$$

- Un attribut à grande échelle, comme la superficie, peut éclipser les attributs à petite échelle, comme le nombre de pièces. KNN « ignore » alors presque les dimensions à petite échelle.
- La **standardisation** (scores z) ou la **mise à l'échelle min-max** place les dimensions sur des échelles comparables.
- **Règle pratique :** pour une méthode fondée sur les distances, mettez les attributs à l'échelle, sauf si leurs unités actuelles représentent délibérément l'importance relative que la distance doit leur accorder.

Distances aux exemples d'entraînement

Distances de **a** à {**b**, **c**} avant et après la mise à l'échelle.

```
def show_pair(name_from, names_to, D):
    return D.loc[name_from, names_to].to_frame("distance")

print("Distances sans mise à l'échelle de a vers {b,c}")
display(show_pair("a", ["b", "c"], dist_unscaled))

print("Distances après mise à l'échelle de a vers {b,c}")
display(show_pair("a", ["b", "c"], dist_scaled))
```

`Distances sans mise à l'échelle de a vers {b,c}`

	distance
b	20.396078
c	200.000000

`Distances après mise à l'échelle de a vers {b,c}`

	distance
b	2.008247
c	1.818182

Et la distance de Manhattan ?

L'échelle importe également avec la distance L_1 :

$$\|u - v\|_1 = \sum_j |u_j - v_j|.$$

Les mêmes données d'entraînement peuvent être réutilisées avec `metric="manhattan"`.

```
# 1) Sans mise à l'échelle
knn_manhattan_plain = KNeighborsRegressor(
    n_neighbors=1, metric="manhattan"
)
knn_manhattan_plain.fit(X_train, y_train)
pred_manhattan_plain = knn_manhattan_plain.predict(X_query)[0]

# 2) Avec mise à l'échelle
knn_manhattan_scaled = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=1, metric="manhattan"))
])
knn_manhattan_scaled.fit(X_train, y_train)
pred_manhattan_scaled = knn_manhattan_scaled.predict(X_query)[0]

pd.DataFrame(
    {
```

```
        "prédiction (sans mise à l'échelle)": [pred_manhattan_plain],
        "prédiction (avec mise à l'échelle)": [pred_manhattan_scaled],
    },
    index=["a"]
)
```

	prédiction (sans mise à l'échelle)	prédiction (avec mise à l'échelle)
a	520000.0	390000.0

En résumé

— Les **modèles fondés sur les distances** sont sensibles aux unités et aux échelles relatives de leurs attributs.
— Ajustez un outil de mise à l'échelle uniquement sur les **données d'entraînement**, puis réutilisez-le pour les données de validation, de test et de production.
— Un pipeline regroupe la mise à l'échelle et la prédiction et prévient les fuites durant la validation croisée.
— La mise à l'échelle modifie la géométrie du modèle et peut donc changer les plus proches voisins et les prédictions.